



EASYGO

———— Group 1 ————

Andy Thiang

Justin Tan Shao Yang

Dinh Quang Anh

Caasi John Rian Domingo

Benjamin Phang Qi Lun

Chew Hong Ming

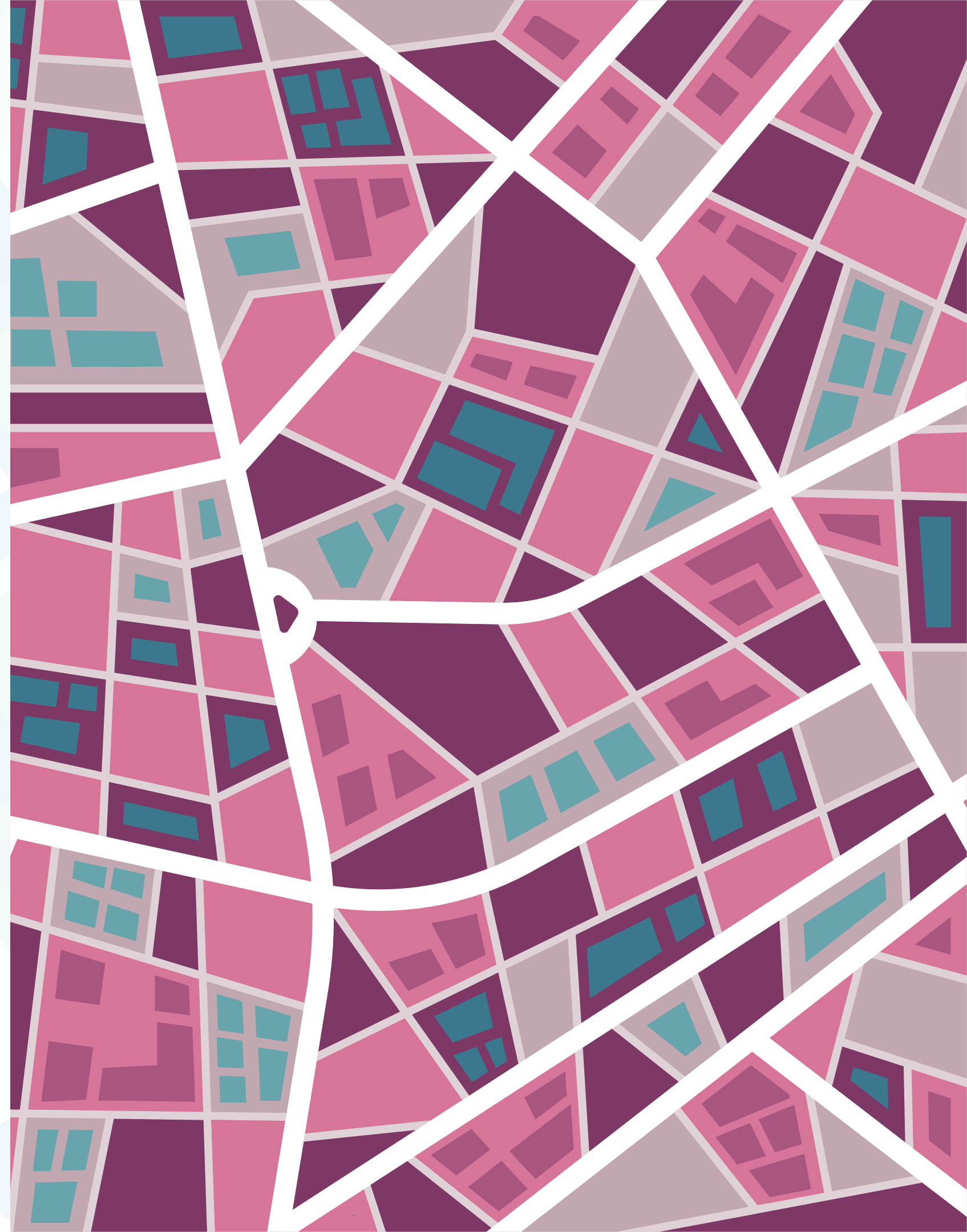


INNOVATION OVERVIEW

- EasyGo is a digital solution that generates **optimised** travel routes tailored to individual traveler preferences. It shifts the navigation from fixed outputs to dynamic, user controlled experience.
- The originality is the **Dynamic Convenience Score**, a metric that allows the user to personally weigh the importance of factors like walking, crowds and cost. EasyGo provides a clear, actionable comparison of trade-offs which leads to maximum user satisfaction with every journey.

Problem Statement

- I. Commuters in Singapore often juggle between **multiple applications** to plan a single journey (e.g. one app for route, another for public transit information)
- I. Existing route planners typically present the fastest or shortest route as the “best” option, failing to account for **individual user priorities**



Tech Stack

Frontend



Backend



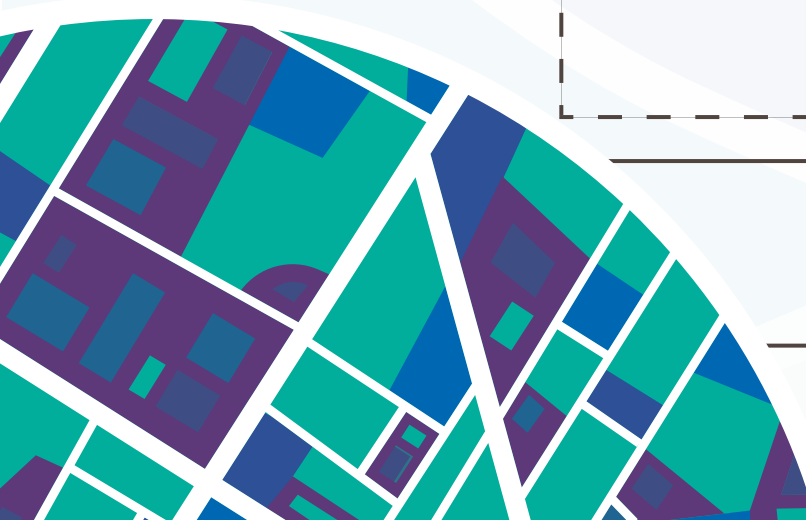
DATA.GOV.SG



FullStack/ Database

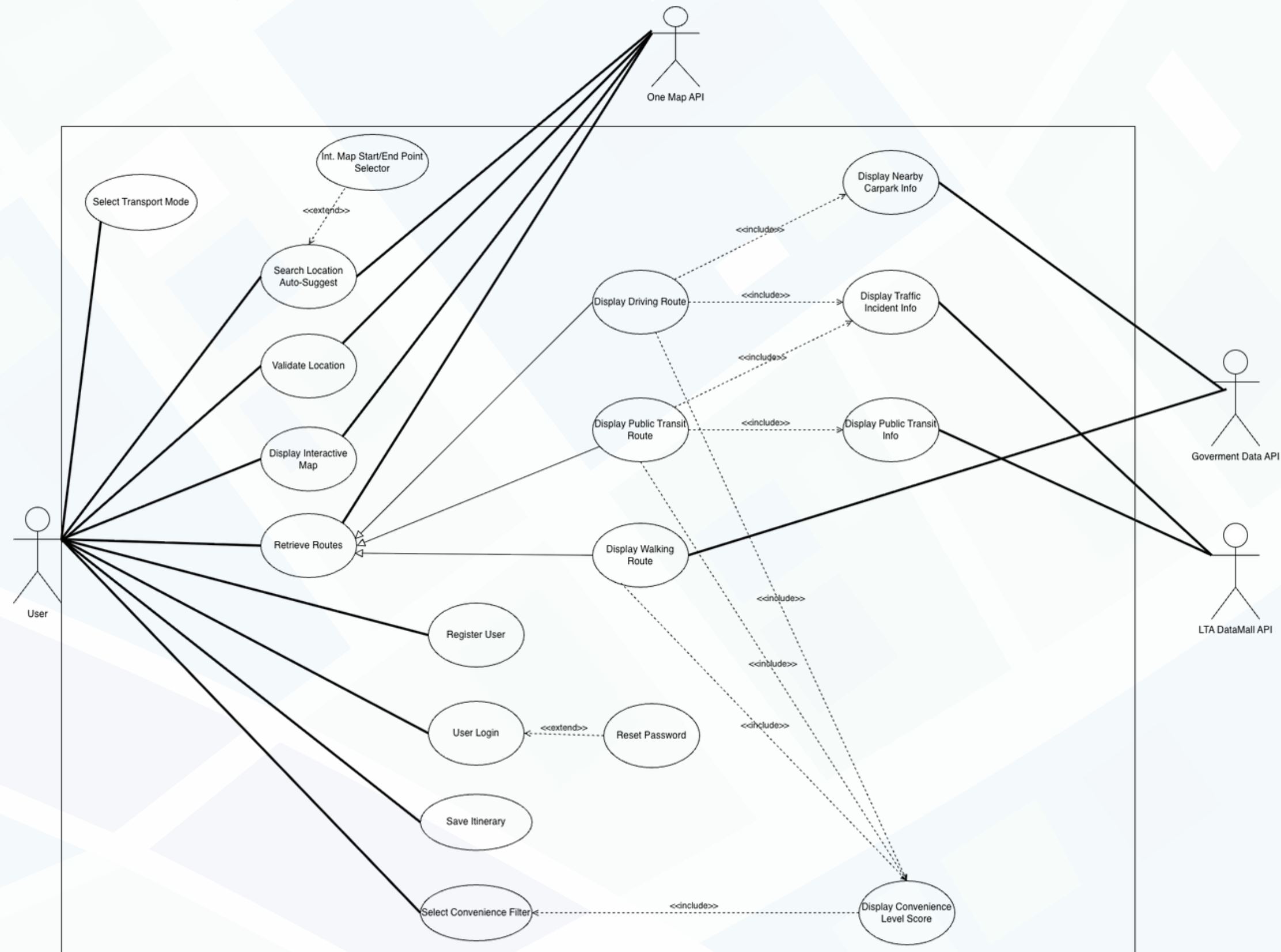


Key Features



Authentication	Traffic Incidents Info
Settings/Profile	Public Transit Info
Location Input	Weather Forecast
Live Map	Multi-factor Convenience Score
Route Generation	Scoring Weights
Save Routes	Notifications
Carpark Info	API Usage

Use Case Diagram



Feature List

1. Multi-Factor Convenience Score

- Indicates how convenient a route is based on user preference
- Allows user to instantly **compare route quality** against personal priorities

2. Scoring Weights

- User can adjust the importance (weights) of factors to generate routes that reflect **specific needs** for users journey

3. Carpark Info

- Solves the final mile issue for car users
- **Showcases available carparks nearby** integrated into convenience score

Feature List

3. Authentication

- Login
- Register

4. Location Input

- Search bar to input **start and end location**
- Search bar can **auto-complete** the location inputs
- Location can be selected by **clicking on the map**

5. Route Generation

- Generates routes dedicated to mode of transport
- Has a **"Best" route** for the user based on personal preferences

6. Live Map

- Shows the respective route based on user chosen itinerary.
- Traffic incident will be displayed in the map

Feature List

7. Saved Routes

- Save itinerary under **Saved Routes Tab**
- View Saved Routes

8. Weather Forecast

- Can be viewed for **walking routes**

9. Public Transit Info

- Provide information like **number of transfers**, congestions at platforms etc.

10. Traffic Incident Info

- Essential for reliable routing.
- Provides updates on **accidents and road closures**

External APIs

1. OneMap API

- a. Live Map
- b. Geocoding

2. Data.gov.sg

- a. Weather

3. LTA DataMall

- a. CarPark Availability
- b. Platform Density
- c. Traffic Incident
- d. Bus Arrival Time



The background is a complex, abstract geometric pattern. It features a central area with a light blue and white grid-like pattern, surrounded by darker blue and green shapes. The overall design is modern and minimalist, with a focus on geometric forms and a limited color palette of blue, green, and orange.

Live Demo!

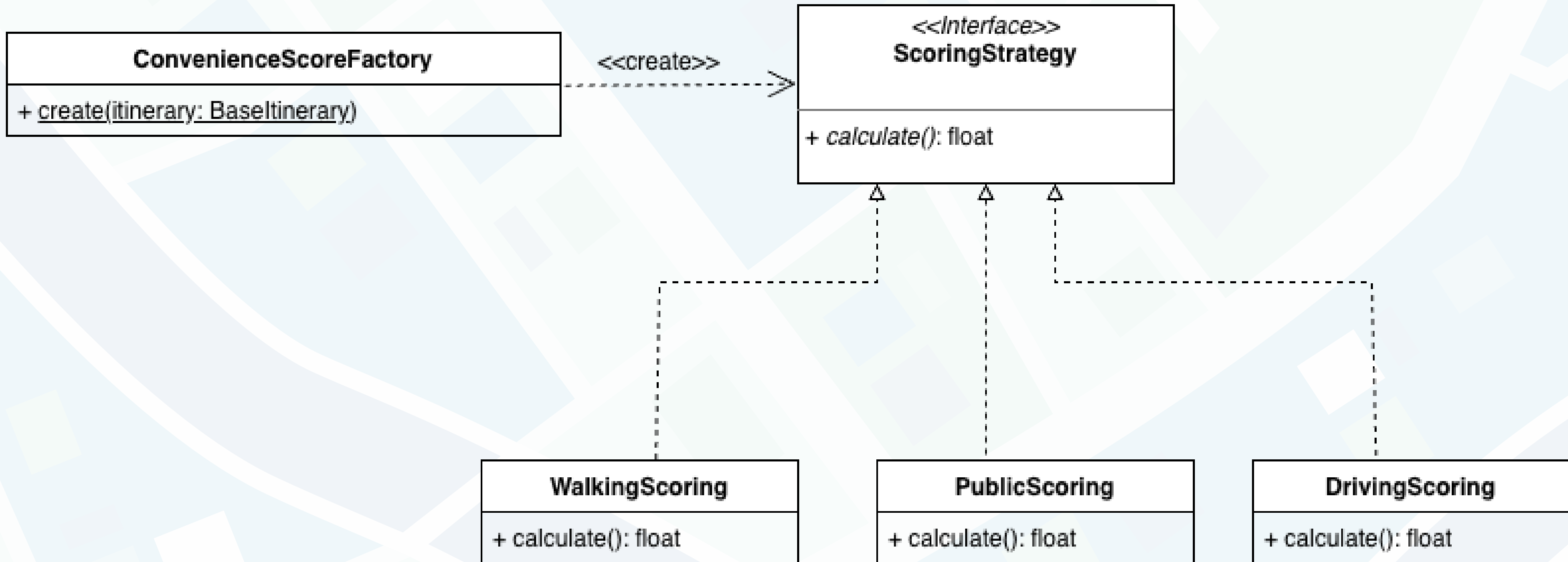


Software Engineering Practices and Design Patterns

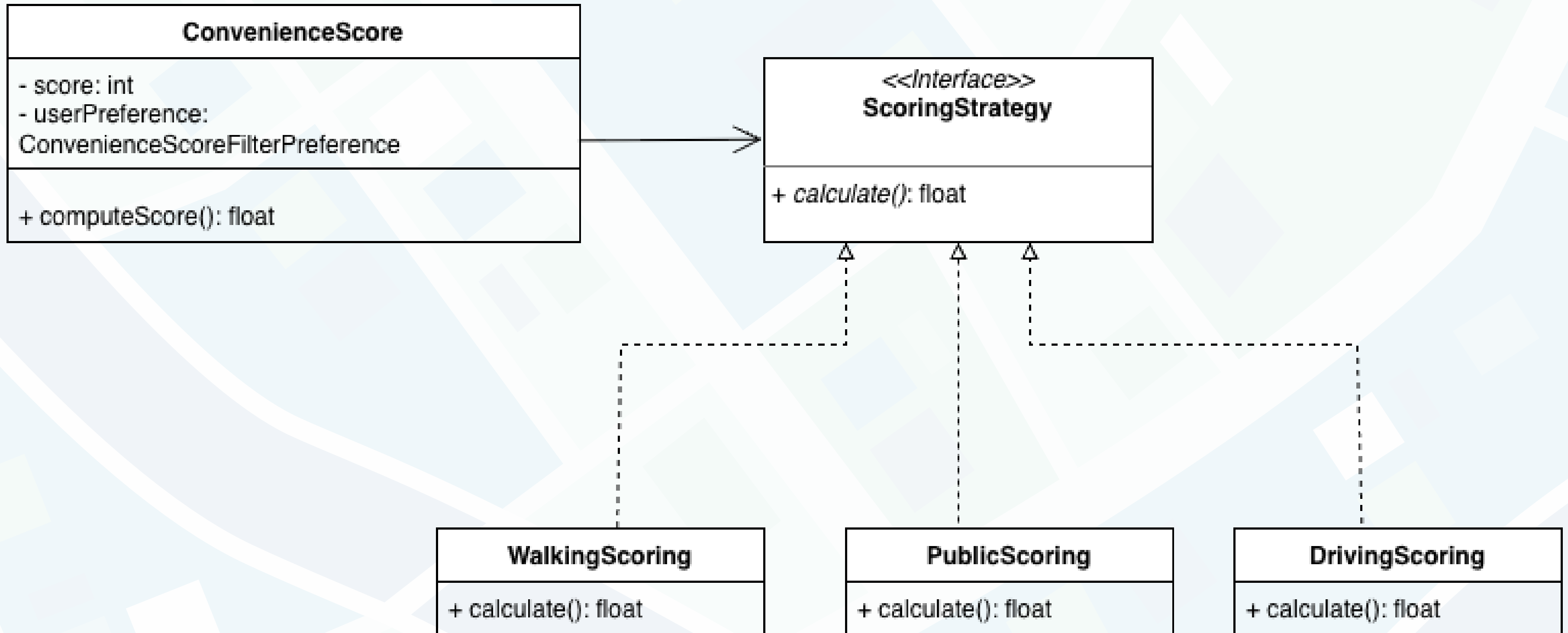


Design Patterns

Factory Pattern



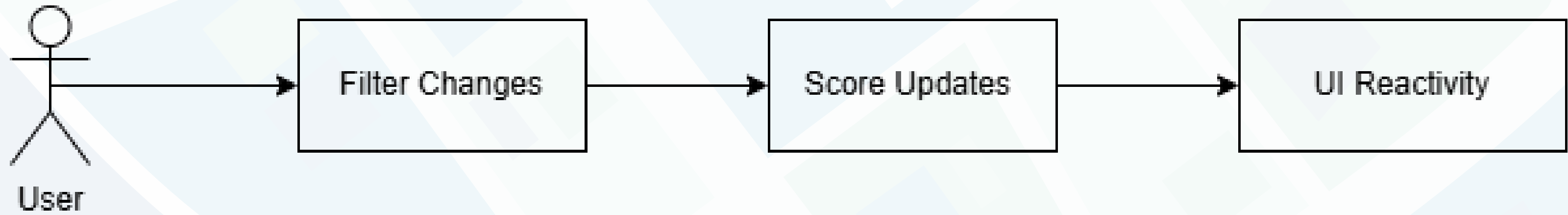
Strategy Pattern



Observer

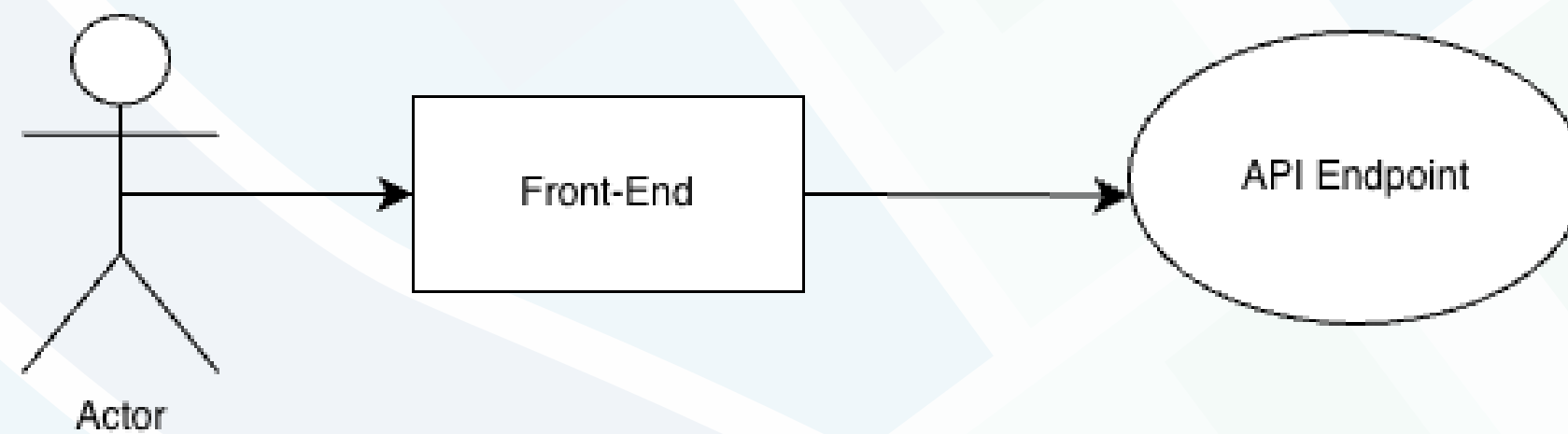
Observer Pattern

Observer Pattern

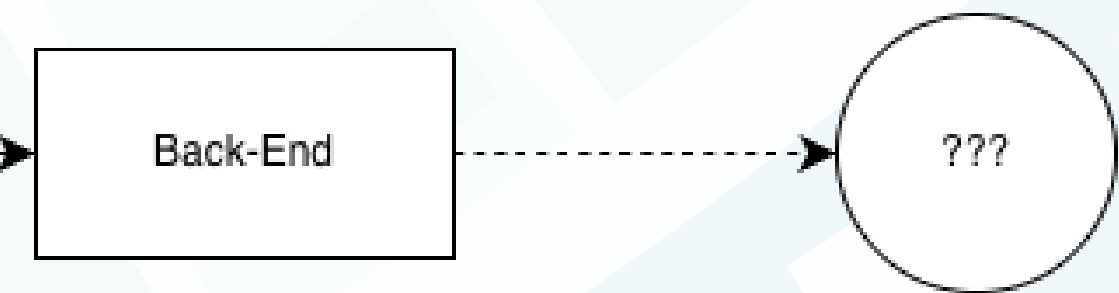


Facade

Facade Pattern



Future Extensionability



Design Principles(SOLID)

<i>Single Responsibility Principle (SRP)</i>	Each API route handles one specific job	• Easier debugging and error handling • More reusability
<i>Open-Closed Principle (OCP)</i>	Add new route type without touching core logic	• Prevent merge conflicts (More Scalable) • Reduce risk of breaking code(closed for modification) • Open for extension
<i>Dependency Inversion Principle (DIP)</i>	High level and low level components (controllers etc..) depends on abstractions through the use of the interfaces	• Cleaner code and easier for teams to work on simultaneously. • Flexible



Good SWE Practices

Documentation – README

EasyGo

Welcome to the official repository for NTU SC2006 / SC2002 Software Engineering group project *EasyGo*.



[Self Hosted Live Demo](#)

EasyGo is a smart transport and convenience planner that helps users find the most optimal route based on overall comfort over just speed.

By integrating real-time public transport, bus wait timings, carpark... data, EasyGo computes a personalized Convenience Score that balances time, cost, walking distance, and live conditions.

Built with Next.js + TypeScript, with scalability, modularity, and strong software engineering practices in mind.

- Live Demo
- Supporting Documents
- Diagrams

Project Overview

Goal:

To develop a web application that consolidates multiple transport and environmental data sources to compute a “Convenience Score”, helping users select routes that best fit their preferences and live conditions.

Features

Category	Description
Authentication	Supports Driving, Public Transport, and Walking via the OneMap Routing API.
Convenience Scoring System	Calculates a customized score based on weighted user preferences (time, walking distance, fare cost, carpark availability, etc.).
Real-Time Data Integration	Uses OneMap, LTA DataMall, and NEA Weather APIs for live conditions.
Interactive Map Visualization	Visualized using Leaflet.js with custom popups, carpark overlays, and selectable route layers.
Route Generation	Generate routes which is retrieved from oneMap API
Route Specific Information Display	Displays information specific to route (traffic incident / weather).
Save Route / Locations	Saves locations for easy access

Future guide for developers, to ensure long-term consistency in developments

Documentation – Comments

```
// get bus wait time and platform density for public itineraries
await Promise.all(
  publicItineraries.map(async i => {
    try {
      await controller.getBusWaitTime(i);
      await controller.getPlatformDensity(i);
    } catch (e) {
      console.error(e);
    }
  })
);
// get incident and weather for driving itineraries
await Promise.all(
  drivingItineraries.map(async i => {
    try {
      await controller.getWeatherData(i);
      await controller.getTrafficIncidents(i);
    } catch (e) {
      console.error(e);
    }
  })
);
```

For better comprehension and understanding

Strong Typing

```
return this.parseSimpleRoute(json as OneMapDrivingRouteResponse, mode)
```

```
export interface OneMapDrivingRouteResponse {  
  status_message: string  
  route_geometry: string  
  status: number  
  route_instructions: RouteInstruction[]  
  route_name: string[]  
  route_summary: RouteSummary  
  alternativeroute?: AlternativeRoute[]  
  viaRoute?: string  
  subtitle?: string  
  phyroute?: PhysicalRoute  
}
```

Since we are using typescript, we must always avoid using “any” unless its for well-justified fallback cases

Code Style



Ensures Code Quality



Enforces Coding Style and Consistency (Linting)



Identify Problematic Patterns



Prettier

Reusability and Refactoring

```
8  ✓ export interface MapDisplayHandle {  
9      map: L.Map  
10     panTo: (lat: number, lng: number, popupText?: string) => void  
11     clearPolylines: () => void  
12 }
```

define the MapDisplayHandle (the reusable public API the parent can call)

```
21     const MapDisplay = forwardRef<MapDisplayHandle, MapDisplayProps>(  
22         ({ setLayoutInURL, setStartValue, setEndValue }, ref) => {  
23             const mapContainerRef = useRef<HTMLDivElement | null>(null)  
24             const mapRef = useRef<L.Map | null>(null)  
25             const markerRef = useRef<L.Marker | null>(null)
```

component is wrapped with forwardRef so a parent may attach a ref

```
165         useImperativeHandle(ref, () => ({  
166             map: mapRef.current as L.Map,  
167  
168     ✓     panTo(lat, lng, popupText) {
```

```
175         mapRef.current.setView([lat, lng], 18);  
176         markerRef.current = marker;  
177     },  
178     ✓     clearPolylines() {
```

useImperativeHandle exposes only the small set of imperative methods (map getter, panTo, clearPolylines)

Reusability and Refactoring

```
"use client"

import { useState } from "react"
import {
  ItineraryData,
  DrivingItineraryData,
  PublicItineraryData,
  WalkingItineraryData,
  BaseItineraryData,
} from "@lib/controllers/Parser"

export interface GetItinerariesResponse {
  driving: ItineraryData<DrivingItineraryData>[]
  pt: ItineraryData<PublicItineraryData>[]
  walking: ItineraryData<WalkingItineraryData>[]
}

export interface GetScoreResponse {
  best: { score: number; itinerary: ItineraryData<BaseItineraryData> }[]
  driving: { score: number; itinerary: ItineraryData<DrivingItineraryData> }[]
  public: { score: number; itinerary: ItineraryData<PublicItineraryData> }[]
  walking: { score: number; itinerary: ItineraryData<WalkingItineraryData> }[]
}

export interface ConvenienceFilter {
  durationWeight: number
  walkingDistanceWeight: number
  noTransferWeight: number
  carparkAvailabilityWeight: number
  busWaitTimeWeight: number
  platformDensityWeight: number
  fareWeight: number
}
```

```
export function useItineraryData() {
  const [routes, setRoutes] = useState<GetScoreResponse | null>(null)
  const [loading, setLoading] = useState(false)

  async function getItineraries(start: [number, number], end: [number, number], s
    const url = `/api/get-itineraries?startLat=${start[0]}&startLon=${start[1]}`
    const res = await fetch(url)
    if (!res.ok) throw new Error(`Failed to fetch itineraries: ${res.status}`)
    return (await res.json()) as GetItinerariesResponse
  }

  async function getScore(
    itineraries: GetItinerariesResponse,
    filters: ConvenienceFilter
  ) {
    const requestbody = {
      filterData: filters,
      driving: itineraries.driving,
      pt: itineraries.pt,
      walking: itineraries.walking,
    }

    const res = await fetch("/api/get-score", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(requestbody),
    })
    if (!res.ok) throw new Error(`Failed to fetch scores: ${res.status}`)
    return (await res.json()) as GetScoreResponse
  }
}
```

hook centralises itinerary fetching + scoring + transformation (reuse)

Reusability and Refactoring

```
import type { BaseItinerary } from "./BaseItinerary";
import type { DrivingItinerary } from "./DrivingItinerary";
import type { PublicItinerary } from "./PublicItinerary";
import type { SimpleWalkingItinerary } from "./SimpleWalkingItinerary";
import { ConvenienceScoreFilterPreference } from "./ConvenienceScoreFilterPreference";

interface MaxMin {
  max: number;
  min: number;
}

function getMaxMin<T extends BaseItinerary>({
  itineraryList: T[],
  selector: (itinerary: T) => number
}): MaxMin {
  const maxMin: MaxMin = {
    max: -Infinity,
    min: Infinity
  };
  itineraryList.forEach(i => {
    if (selector(i) > maxMin.max)
      maxMin.max = selector(i);
    if (selector(i) < maxMin.min)
      maxMin.min = selector(i);
  })
  return maxMin;
}
```

```
export class ConvenienceScore {
  private itinerary: BaseItinerary;
  private strategy: ScoringStrategy;
  private score: number;

  public constructor(itinerary: BaseItinerary, strategy: ScoringStrategy) {
    this.itinerary = itinerary;
    this.strategy = strategy;
    this.score = 0;
  }

  public computeScore(bound: Bound, userPreference: ConvenienceScoreFilterPreference): void {
    const normalizedDurationScore = normalize(this.itinerary.totalDuration, bound.duration);
    const normalizedWalkingDistanceScore = normalize(this.itinerary.walkingDistance, bound.walkingDistance);
    this.score = userPreference.durationWeight * (1 - normalizedDurationScore)
      + userPreference.walkingDistanceWeight * (1 - normalizedWalkingDistanceScore);
    this.score = this.strategy.calculate(this.score, this.itinerary, userPreference, bound);
  }

  public setScore(score: number): void {
    this.score = score;
  }

  public getScore(): number {
    return this.score;
  }
}
```

normalization, Bound, and ConvenienceScore encapsulate scoring logic and make it reusable across itinerary types (reuse)

computeScore is a single place to change scoring behavior (refactor).

SCRUM

SCRUM PROCESS

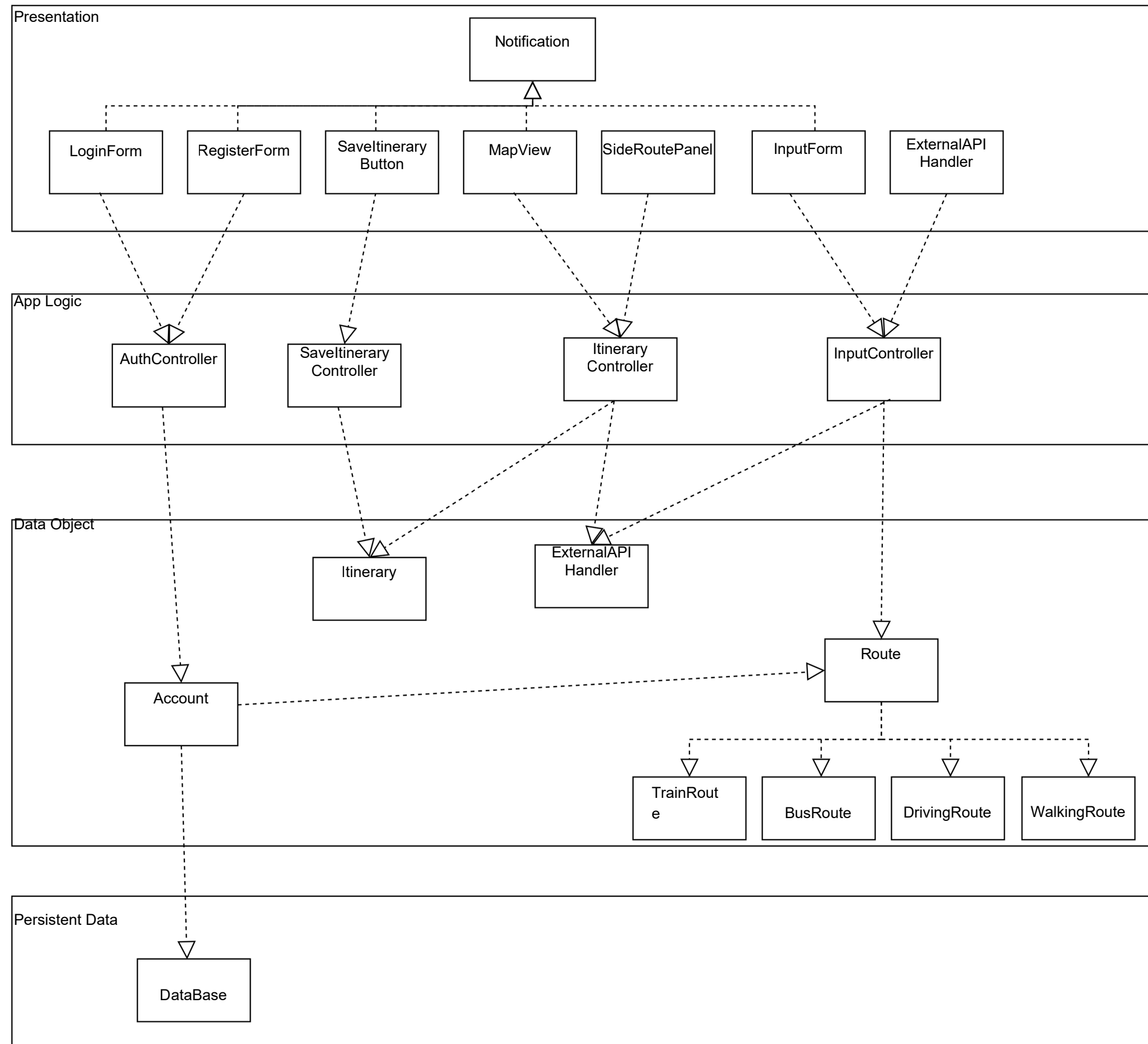


Clear Transparency and Visibility among the team

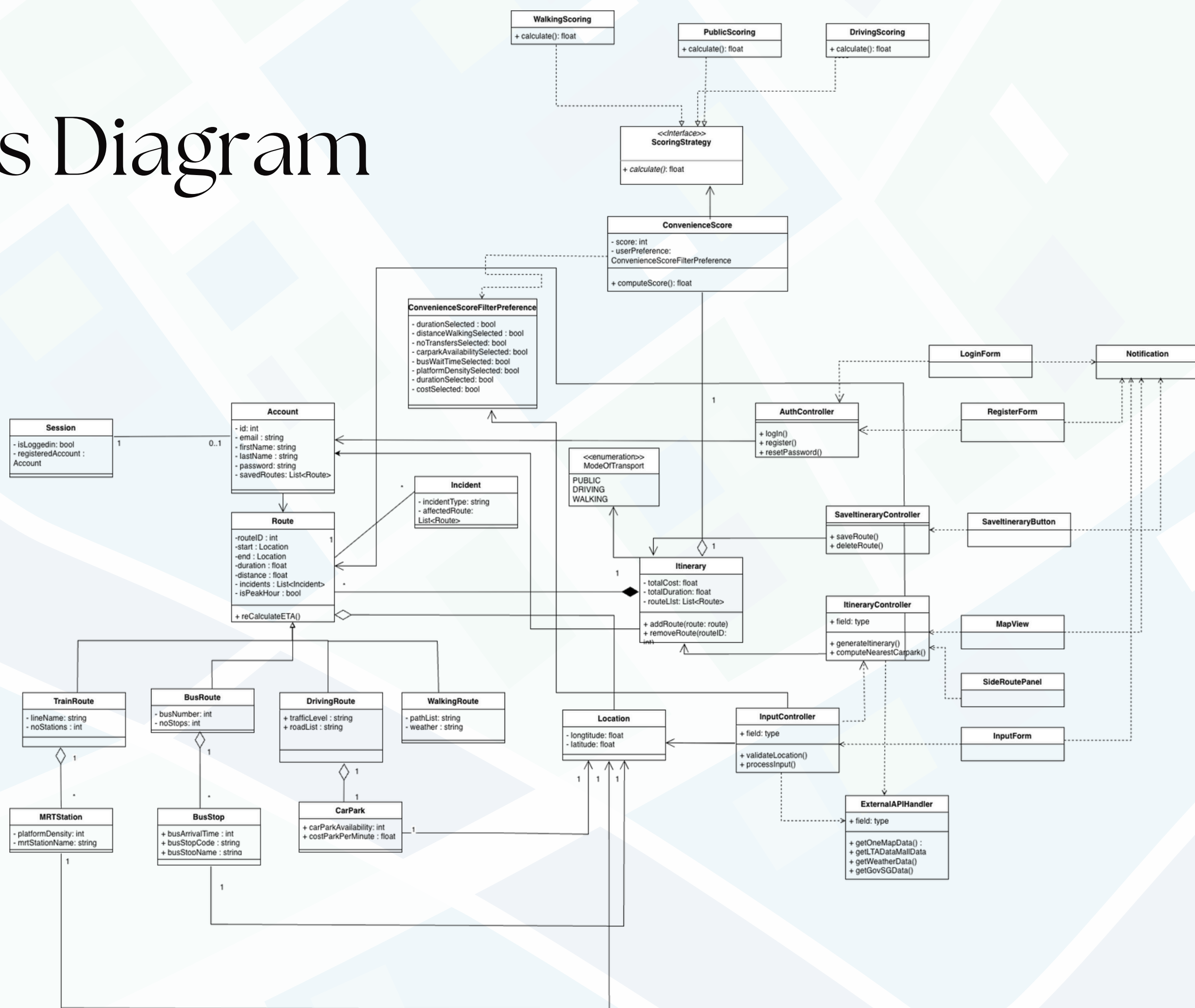
Flexible Product Backlog

Faster Delivery (2 - 4 weeks)

System Architecture



Class Diagram





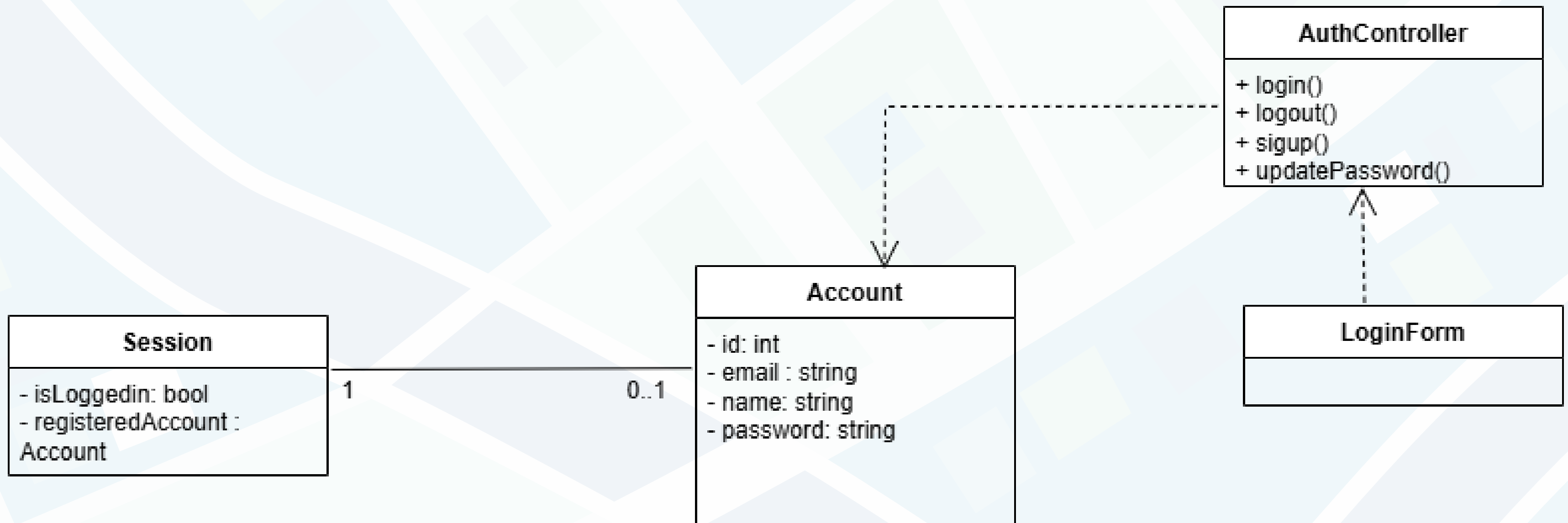
Traceability In Project Deliverables

Use Case Description

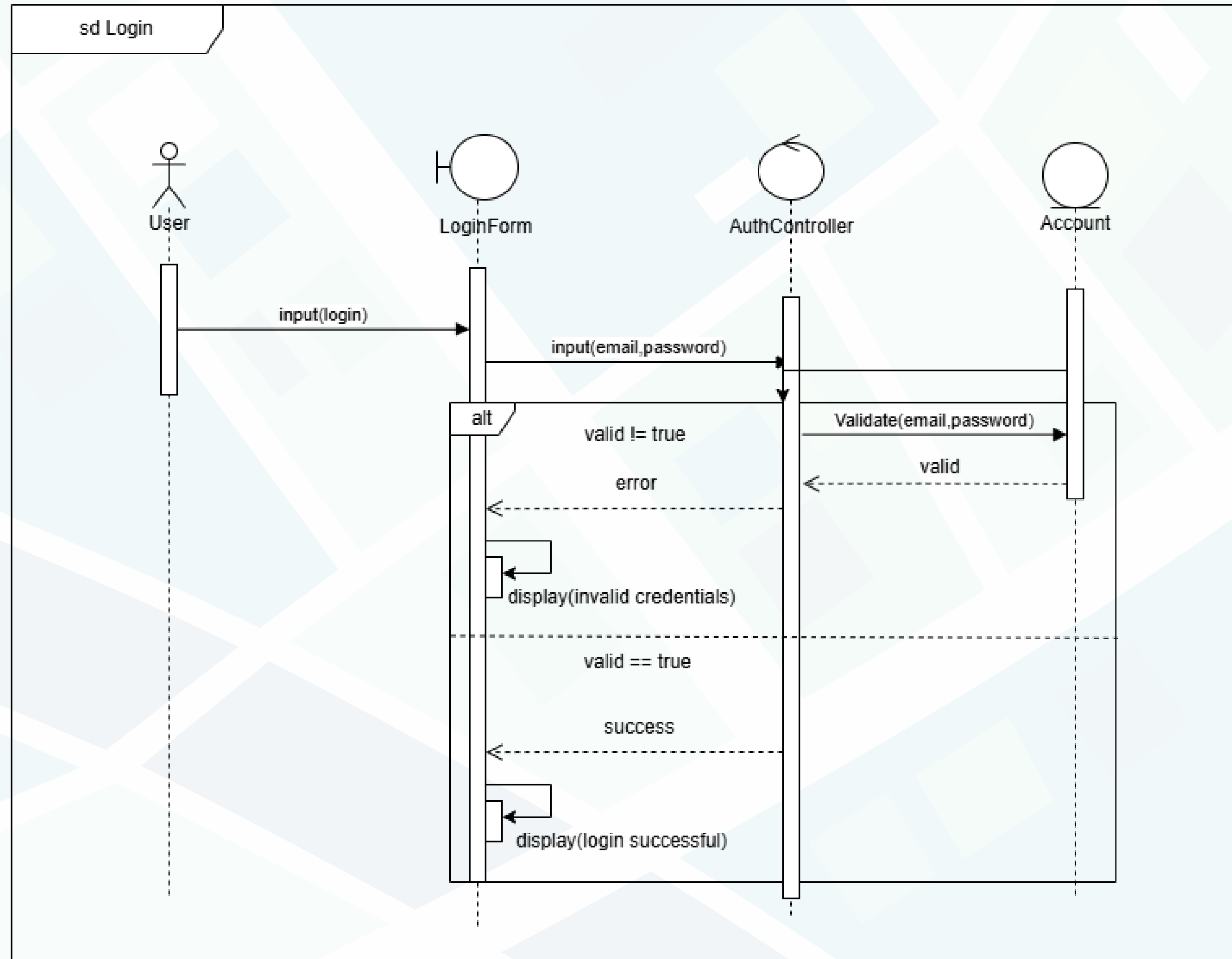
Use Case ID :	5.1		
Use Case Name:	User Login		
Created by:	Caasi John	Last updated by:	Caasi John
Date	25th August 2025	Date Last updated:	25th August 2025
Actor:	User		
Description	The back and front end system involved in managing authentication services within the application. It is used in validating entered credentials and providing appropriate error messages otherwise		
Preconditions	1. User must already have an existing account within application database 2. The system is already connected to the account database		
Postconditions	1. User is assigned a login session token by the authentication service 2. The user is displayed an error message upon input of invalid credentials not present in the application's database		
Priority	High		
Frequency of Use	High		
Flow of Events	1. Unregistered user will be displayed login on the landing page 2. The system pops up a login form with email and password and a login and a forget password button 3. The system validates the input credentials (e.g valid email, password length) 4. The system queries and validates with the account database to check account validity 5. If account is valid, the system provides a login session token and redirects the user to the main page		
Alternative Flows	None		

Exceptions	1. Invalid Login Credentials - If the user provides invalid login credentials, an error message with the text "invalid login credentials, please try again" message below the associated fields 2. Attempt Lockout - After 5 failed login attempts, the account is temporarily locked and an email is sent to the user's email on suspicious login activity 3. Authentication Service is Down - Due to maintenance or other related issues pertaining to the attached credential database, login services will be disabled and will display an error message. "Login services are unavailable at the moment, please try again later".
Includes	5.2
Special Requirements	1. The application in production must support HTTPS and have valid authority certificates. 2. The password in transit and in storage must be salted and hashed for data security
Assumptions	1. The user has an existing valid email
Notes and Issues	Optionally implement Google / external oAuth for login instead of creating an account with our system

Relevant Class Diagram



Relevant Sequence Diagram



Testing and Validation

Black Box and White Box Testing

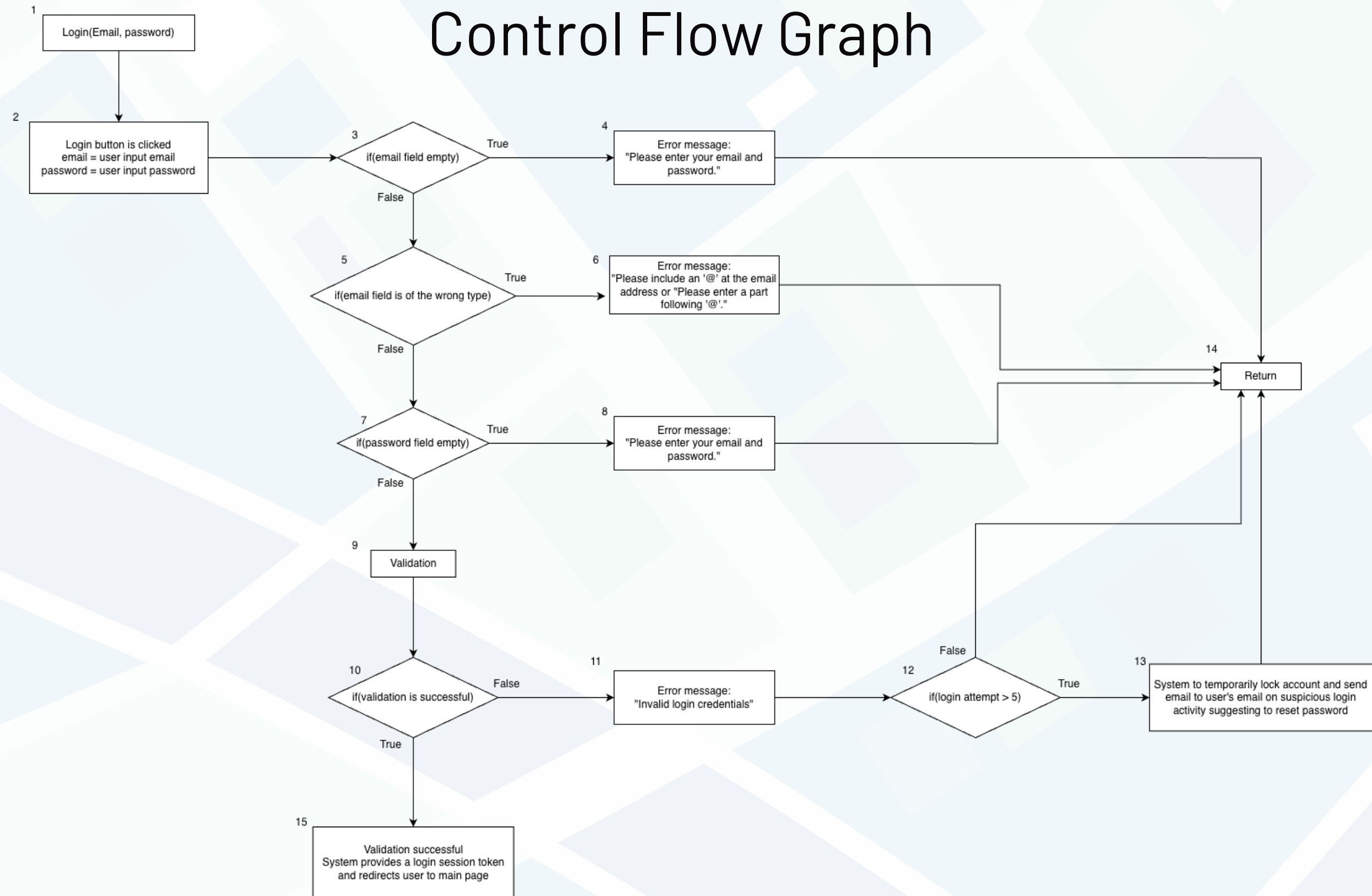
Login(Email, Password)

No.	Test Input	Expected Output	Actual Output	Pass?
Baseline	Email = "Admin1@gmail.com" Password = "12341234!"	"Log in successfully"	"Log in successfully"	Yes
1	Email = "" Password = "12341234!"	"Please enter your email and password!"	"Please enter your email and password!"	Yes
2	Email = "Admin1" Password = "12341234!"	"Please include an '@' at the email address."	"Please include an '@' at the email address."	Yes
3	Email = "Admin1@" Password = "12341234!"	"Please enter a part following '@'."	"Please enter a part following '@'."	Yes
4	Email = "Admin1@gmail.com" Password = ""	"Please enter your email and password!"	"Please enter your email and password!"	Yes
5	Email = "someone1@gmail.com" Password = "12341234!"	"Invalid login Credentials"	"Invalid login Credentials"	Yes
6	(5th Attempt incorrect) Email = "someone1@gmail.com" Password = "12341234!"	"Account is locked, please check your email to reset your password~"	"Account is locked, please check your email to reset your password~"	Yes

Testing and Validation

Black Box and White Box Testing

Control Flow Graph



Testing and Validation

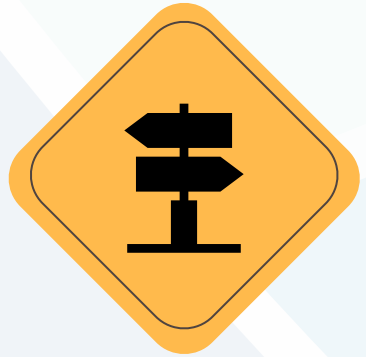
Black Box and White Box Testing Basis Paths

Cyclomatic Complexity = | decision points | + 1 = 5 + 1 = 6

Basis paths

1. Baseline Path: 1, 2, 3, 5, 7, 9, 10, 15
2. Basis Path 1: 1, 2, 3, 4, 14
3. Basis Path 2: 1, 2, 3, 5, 6, 14
4. Basis Path 3: 1, 2, 3, 5, 7, 8, 14
5. Basis Path 4: 1, 2, 3, 5, 7, 9, 10, 11, 12, 14
6. Basis Path 5: 1, 2, 3, 5, 7, 9, 10, 11, 12, 13, 14

Future Plans



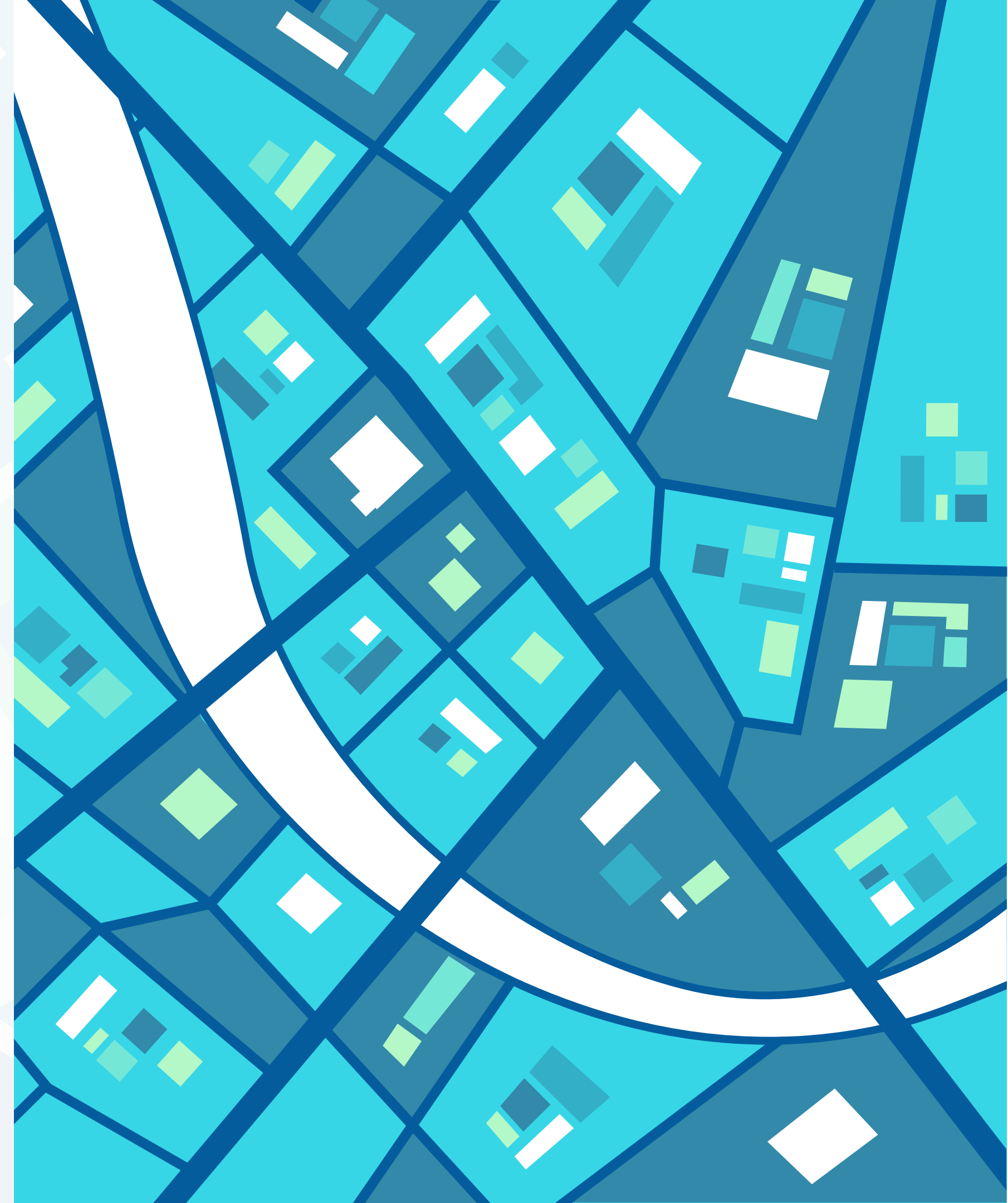
AI-Integrated Predictive
Modelling, EV Charging



Intermediary Stops, Ride-
Sharing Integration



Geographical Expansion,
Accessibility Features, Mobile
Native App





Thank you!